Phase 3: Python Ingestion Layer Completion Report

1. Phase Overview

Phase 3 of the Banking Data Governance & Data Quality Pipeline focused on building a Python-based ingestion layer.

The goal of this phase was to extract all PostgreSQL source and control tables created in Phase 2 and land them into a local raw data zone as Parquet files. This phase represents the first data movement step in the pipeline, moving data from the simulated banking source system into a raw analytical landing area.

The purpose of Phase 3 was to practice practical data engineering skills related to Python scripting, database connectivity, raw data ingestion, file-based data lake design, logging, error handling, and basic ingestion validation.

Phase 3 did not include PySpark transformation, curated warehouse modeling, SQL data quality validation, OpenMetadata setup, governance YAML artifacts, lineage documentation, or Agile/JIRA simulation. Those items belong to later phases.

---

2. Phase 3 Objective

The objective of Phase 3 was:

To extract all 10 PostgreSQL banking source/control tables into a local raw Parquet zone using Python, with load-date partitioning, logging, row-count reporting, and basic validation.

Expected raw output pattern:

data/raw/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

Example:

data/raw/customers/load_date=2026-06-16/customers.parquet

data/raw/accounts/load_date=2026-06-16/accounts.parquet

data/raw/transactions/load_date=2026-06-16/transactions.parquet

---

3. Phase 3 Scope

Phase 3 included only the Python ingestion layer.

In scope:

- Python extraction scripts
- PostgreSQL connectivity
- Raw Parquet file generation

- Load-date partitioning
- Basic logging
- Error handling
- Row-count reporting
- Parquet read-back verification
- README command updates
- Git commit and phase tag

Out of scope:

- PySpark processing
- SQL data quality validation
- OpenMetadata setup
- Governance YAML artifacts
- Data catalog artifacts
- Lineage mapping
- Curated warehouse modeling
- Fact and dimension tables
- Agile/JIRA simulation artifacts

This strict scope helped keep the project disciplined and aligned with SDLC-style phased delivery.

---

4. Tables Extracted

The following 10 PostgreSQL source/control tables were extracted:

1. branches
2. products
3. reference_codes
4. customers
5. accounts
6. transactions
7. customer_risk_scores
8. data_quality_rules

9. stewardship_issues
10. source_to_target_mappings

These tables were previously created and populated in Phase 2.

---

5. Phase 3 Repository Structure

Phase 3 added the following ingestion structure:

```
python/
  ingestion/
    db_connection.py
    extract_postgres_to_raw.py
```

Runtime-generated folders:

```
data/
  raw/


logs/
  ingestion/
```

The runtime folders are generated locally when the ingestion script runs. They are intentionally ignored by Git and are not committed to the repository.

---

6. Files Created or Updated

New Files Created

python/ingestion/db_connection.py

python/ingestion/extract_postgres_to_raw.py

Files Updated

requirements.txt

README.md

.gitignore

Runtime Files Generated Locally

data/raw/

logs/ingestion/

The runtime raw Parquet files and ingestion logs were not committed.

---

7. Python Ingestion Components

7.1 db_connection.py

The db_connection.py file handles PostgreSQL connection creation.

Main responsibilities:

- Load environment variables from .env
- Validate required PostgreSQL configuration
- Create a SQLAlchemy engine
- Avoid logging sensitive credentials
- Keep database connection logic separate from extraction logic

Required environment variables:

POSTGRES_HOST

POSTGRES_PORT

POSTGRES_DB

POSTGRES_USER

POSTGRES_PASSWORD

This separation makes the ingestion code cleaner and easier to maintain.

7.2 extract_postgres_to_raw.py

The extract_postgres_to_raw.py file performs the actual extraction from PostgreSQL to raw Parquet.

Main responsibilities:

- Connect to PostgreSQL using the shared connection utility
- Extract selected tables from PostgreSQL
- Write each table to local Parquet format
- Use load-date partitioned output folders
- Log extraction progress
- Generate a summary CSV

- Validate table names
- Validate load date format
- Verify Parquet row counts after writing
- Exit with failure if validation fails

Supported command-line arguments:

--load-date

--output-dir

--log-dir

--tables

Default behavior:

- Extract all approved source/control tables
- Use today's date as the load date if no load date is provided
- Write raw files to data/raw
- Write logs to logs/ingestion

---

8. Raw Output Design

The raw zone uses local Parquet files partitioned by load date.

Output pattern:

data/raw/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

Example output paths:

data/raw/branches/load_date=2026-06-16/branches.parquet

data/raw/products/load_date=2026-06-16/products.parquet

data/raw/reference_codes/load_date=2026-06-16/reference_codes.parquet

data/raw/customers/load_date=2026-06-16/customers.parquet

data/raw/accounts/load_date=2026-06-16/accounts.parquet

data/raw/transactions/load_date=2026-06-16/transactions.parquet

data/raw/customer_risk_scores/load_date=2026-06-16/customer_risk_scores.parquet

data/raw/data_quality_rules/load_date=2026-06-16/data_quality_rules.parquet

data/raw/stewardship_issues/load_date=2026-06-16/stewardship_issues.parquet

data/raw/source_to_target_mappings/load_date=2026-06-16/source_to_target_mappings.parquet

This design simulates a common raw data lake landing pattern where each load is partitioned by ingestion date.

---

9. Phase 3A: Baseline PostgreSQL to Raw Parquet Ingestion

Phase 3A Goal

Phase 3A created the baseline ingestion layer.

The goal was to extract all 10 PostgreSQL source/control tables into local raw Parquet files.

Phase 3A Work Completed

Phase 3A added:

python/ingestion/db_connection.py

python/ingestion/extract_postgres_to_raw.py

It also updated:

requirements.txt

README.md

.gitignore

Phase 3A Capabilities

Phase 3A implemented:

- PostgreSQL connection through SQLAlchemy
- Environment variable loading with python-dotenv
- Extraction of all 10 approved tables
- Table whitelist validation
- Optional table selection through --tables
- Raw Parquet file generation
- Load-date partitioned folder structure
- Console logging
- File logging
- Runtime summary CSV
- Error handling

- Non-zero failure behavior for extraction failures

Phase 3A Validation

The ingestion script successfully extracted all 10 tables.

Row counts extracted:

Table Name	Row Count
branches	10
products	6
reference_codes	72
customers	500
accounts	990
transactions	5000
customer_risk_scores	742
data_quality_rules	12
stewardship_issues	8
source_to_target_mappings	25

The extracted row counts matched the source data generated in Phase 2.

Phase 3A Commit

Commit:

5a8e5d4 Phase 3A - Add PostgreSQL raw Parquet ingestion

---

10. Phase 3B: Ingestion Hardening

Phase 3B Goal

Phase 3B hardened the ingestion layer with basic production-style validation.

The purpose was to make the ingestion process more reliable before closing Phase 3.

Phase 3B Work Completed

Phase 3B added:

- Load-date format validation

- Parquet read-back verification
- Extracted row count vs Parquet row count comparison
- Enhanced summary CSV
- Improved logging
- README validation command updates

Load-Date Validation

The script now accepts only the following load-date format:

YYYY-MM-DD

Valid example:

2026-06-16

Invalid example:

06-16-2026

Invalid load dates fail with a clear configuration error.

Example error:

Configuration error: --load-date must use YYYY-MM-DD format.

Table Whitelist Validation

The script validates requested table names against the approved source table list.

Invalid table example:

python python\ingestion\extract_postgres_to_raw.py --tables not_a_table --load-date 2026-06-16

Expected error:

Configuration error: Invalid table name(s): not_a_table. Allowed tables: branches, products, reference_codes, customers, accounts, transactions, customer_risk_scores, data_quality_rules, stewardship_issues, source_to_target_mappings

This prevents accidental or unsafe extraction of unapproved database objects.

Parquet Read-Back Verification

After each table is written to Parquet, the script reads the Parquet file back and compares:

extracted_row_count

parquet_row_count

If the counts match, the table is marked as successful.

If the counts do not match, the table is marked as failed and the run exits with a non-zero status.

This confirms that the data written to Parquet is readable and has the expected row count.

Enhanced Summary CSV

The summary CSV now includes:

table_name

status

extracted_row_count

parquet_row_count

output_path

error_message

Example CSV header:

table_name,status,extracted_row_count,parquet_row_count,output_path,error_message

This provides lightweight operational evidence for each ingestion run.

Phase 3B Commit

Commit:

8554b4b Phase 3B - Harden raw ingestion validation

---

11. Commands Used

Compile Python Files

python -m py_compile python\ingestion\db_connection.py python\ingestion\extract_postgres_to_raw.py

Start PostgreSQL

docker compose up -d

Check PostgreSQL Readiness

docker compose exec postgres pg_isready -U banking_user -d banking

Expected output:

/var/run/postgresql:5432 - accepting connections

Run Single-Table Extraction

python python\ingestion\extract_postgres_to_raw.py --tables customers --load-date 2026-06-16

Run Full Extraction

python python\ingestion\extract_postgres_to_raw.py --load-date 2026-06-16

Test Invalid Table Name

python python\ingestion\extract_postgres_to_raw.py --tables not_a_table --load-date 2026-06-16

Test Invalid Load Date

python python\ingestion\extract_postgres_to_raw.py --tables customers --load-date 06-16-2026

View Latest Summary CSV

$latestSummary = Get-ChildItem .\logs\ingestion\postgres_to_raw_summary_*.csv | Sort-Object LastWriteTime -Descending | Select-Object -First 1

Import-Csv $latestSummary.FullName | Format-Table

Check Git Status

git status --short

---

12. Validation Results

Phase 3 validation passed successfully.

Validation confirmed:

- Python files compiled successfully
- PostgreSQL container was running
- PostgreSQL accepted connections
- Single-table extraction succeeded
- Full-table extraction succeeded
- All 10 tables were extracted
- Raw Parquet files were generated under data/raw
- Logs were generated under logs/ingestion
- Summary CSV was generated
- Invalid table names were rejected
- Invalid load dates were rejected
- Parquet row counts matched extracted row counts
- Runtime folders did not appear in Git status
- Final Git status was clean

---

13. Full Extraction Row Count Results

The final full extraction produced the following row counts:

Table Name	Extracted Row Count	Parquet Row Count	Status
branches	10	10	SUCCESS
products	6	6	SUCCESS
reference_codes	72	72	SUCCESS
customers	500	500	SUCCESS
accounts	990	990	SUCCESS
transactions	5000	5000	SUCCESS
customer_risk_scores	742	742	SUCCESS
data_quality_rules	12	12	SUCCESS
stewardship_issues	8	8	SUCCESS
source_to_target_mappings	25	25	SUCCESS

All extracted row counts matched the Parquet read-back row counts.

---

14. Git History After Phase 3

Final Git history after completing and tagging Phase 3:

8554b4b (HEAD -> main, tag: phase-3-complete) Phase 3B - Harden raw ingestion validation

5a8e5d4 Phase 3A - Add PostgreSQL raw Parquet ingestion

ef5944f (tag: phase-2-complete) Phase 2D - Add source data load instructions and compose cleanup

2a03452 Phase 2C - Load normalized synthetic banking source data

f2257ca Phase 2B - Implement normalized PostgreSQL source schema

c3beab3 (tag: phase-1-complete) Phase 1 - Governance pipeline foundation setup

Phase 3 tag:

phase-3-complete

---

15. Final Phase 3 Status

Phase 3 is complete.

Final status:

Item	Status
Phase 3A baseline ingestion	Complete
Phase 3B ingestion hardening	Complete
Raw Parquet generation	Complete
Load-date partitioning	Complete
Logging	Complete
Summary CSV reporting	Complete
Invalid table validation	Complete
Invalid load-date validation	Complete
Parquet row-count verification	Complete
Runtime files ignored by Git	Complete
Git commit completed	Complete
Phase tag created	Complete

---

16. Skills Practiced in Phase 3

Phase 3 helped practice the following skills:

- Python scripting
- PostgreSQL connectivity
- SQLAlchemy engine creation
- Environment-based configuration
- Data extraction automation
- Raw data zone design
- Parquet file generation
- Load-date partitioning

- Logging and operational traceability
- Error handling
- Row-count reconciliation
- Basic ingestion validation
- Command-line argument handling
- Git commit and tagging discipline
- SDLC-style phased delivery

---

17. Interview Explanation

A clear interview explanation for Phase 3 would be:

“In Phase 3, I built the Python ingestion layer for the banking governance pipeline. The source data was stored in PostgreSQL across 10 normalized source and control tables. I created a Python extraction process that connects to PostgreSQL, extracts each approved table, and lands the data into a local raw zone as Parquet files partitioned by load date. I also added operational logging, a summary CSV, table whitelist validation, load-date validation, and Parquet read-back row-count checks to confirm that the extracted row counts matched the written files. This phase gave me hands-on practice with Python scripting, database extraction, raw data landing patterns, error handling, and ingestion validation.”

---

18. Phase 3 Conclusion

Phase 3 successfully created the raw ingestion foundation for the Banking Data Governance & Data Quality Pipeline.

The project now has a clean flow from:

PostgreSQL banking source system

→ Python ingestion layer

→ local raw Parquet zone

This prepares the project for the next phase:

Phase 4: PySpark Processing Layer

In Phase 4, the raw Parquet files will be read using PySpark and processed into standardized clean outputs. Curated warehouse modeling, SQL data quality validation, OpenMetadata integration, and governance artifacts will still remain separate future phases.